

BOUND: Boundary-Oriented Understanding and Editing for Package Hallucination

Abstract—Large language models (LLMs) have demonstrated strong capabilities in software engineering tasks, such as code generation, library recommendation, and dependency configuration. However, recent studies show that LLMs may suffer from package hallucination, where they generate non-existent or invalid package names. These hallucinations can be exploited in software supply chain attacks, as attackers may register malicious packages under hallucinated names. Therefore, mitigating package hallucination is important for improving the reliability and security of LLM-assisted software development. In this paper, we introduce BOUND, a lightweight localized model editing framework for mitigating package hallucinations in LLMs. BOUND formulates package hallucination mitigation as a package-boundary editing problem and aims to refine the model’s internal boundary between valid and hallucinated packages. It first locates modules related to package hallucination through a risk-aware localization strategy, and then edits these modules with lightweight LoRA adapters using a boundary-aware objective that reinforces valid packages, suppresses hallucinated packages, and preserves locality behavior. Experimental results show that BOUND effectively reduces package hallucinations while preserving valid package recommendations. In package recommendation task, BOUND reduces Package-HR by 79.9% on edit prompts and by 65.4% on unseen prompts. The learned package-boundary knowledge further generalizes to other package-related tasks, reducing Package-HR by 10.3% in code generation and by 33.1% in `pip install` recommendation. These results demonstrate that BOUND effectively refines the package-validity boundary of LLMs and improves the reliability of package-related outputs.

I. INTRODUCTION

In recent years, large language models (LLMs) have advanced rapidly and demonstrated strong capabilities across a wide range of software engineering tasks [1], [2], [3], [4]. Developers increasingly rely on LLMs not only to generate code, but also to recommend libraries, configure dependencies, and prepare executable development environments [5], [6]. In these workflows, package names generated by LLMs are often treated as actionable recommendations: they may be copied into import statements, installed through package managers, or consumed by automated coding agents. Consequently, the reliability of package-related outputs has become an important requirement for the practical adoption of LLMs in software development [7], [8].

However, recent studies have shown that LLMs can suffer from hallucinations, generating outputs that appear plausible but are factually incorrect [9], [10], [11]. A particularly concerning type is *package hallucination*, where models generate non-existent, unavailable, or invalid software package names [12], [13], [14]. Unlike many conventional code generation errors, package hallucinations can directly affect depen-

dency management and software supply-chain security [15], [16], [17]. Developers or agents may inadvertently install LLM-generated hallucinated packages, creating opportunities for attackers to register malicious packages under hallucinated names [12]. Such risks are closely related to package confusion attacks, which is a serious threat in supply chain security [18], [19]. As LLM-based coding assistants and autonomous agents become increasingly integrated into software development workflows, mitigating package hallucination is essential for building trustworthy software engineering tools.

Existing studies have proposed various strategies to mitigate package hallucinations, ranging from external mitigation techniques to internal model adaptation approaches [12]. External mitigation methods primarily operate outside the model. One line of work augments generation with external knowledge sources, such as documentation or package repositories (e.g., PyPI [20]) to ground model outputs in reliable package information [21], [22]. Another line of work encourages the model to refine its own responses through self-correction [23], [24]. Although these methods can reduce hallucinations, they do not directly modify the model’s internal behavior and introduce additional inference overhead or pipeline complexity [25]. Internal adaptation methods (e.g., supervised fine-tuning) provide a more direct alternative by modifying the model itself [12], [26]. However, fine-tuning updates a large number of parameters and may overfit to specific output formats, making it computationally expensive and potentially degrading the model’s general capabilities. These limitations motivate the need for a lightweight approach that can directly mitigate package hallucinations of LLMs without extensive retraining or architectural modifications.

In recent years, model editing, also known as knowledge editing, has been proposed to update specific knowledge or behaviors in LLMs without full retraining [27], [28], [29], [30], [31]. This makes it a promising direction for mitigating package hallucination, since the goal is to correct harmful package-related behavior while preserving the model’s general capabilities. However, existing model editing methods primarily focus on factual knowledge updates, such as modifying subject-object associations or correcting individual facts [32], [33], [34], [35]. These methods often rely on clear subject tokens or target phrases to locate and edit the corresponding knowledge. Package hallucination presents a different challenge. Package-related prompts usually contains a complex task descriptions rather than explicit factual subjects, and the desired behavior is not to generate a fixed target output. Instead, the model should suppress non-existent package names while preserving useful

valid packages. Furthermore, valid and hallucinated packages may appear together in the same response, suggesting that the problem is better viewed as correcting a package-validity boundary than rewriting an isolated fact.

In this paper, we introduce **BOUND**, a lightweight localized model editing framework for mitigating package hallucination in LLMs. Unlike existing mitigation methods that rely on external package retrieval or full-model fine-tuning, BOUND aims to refine the package-validity boundary within the model itself. Specifically, BOUND first identifies the modules that are most related to package hallucination through a risk-aware localization strategy, which locates modules whose parameters are most influential to the model’s preference for hallucinated packages over valid ones. BOUND then injects lightweight LoRA adapters into the localized modules and optimizes them with a boundary-aware objective that reinforces valid packages, suppresses hallucinated packages, and preserves the original model behavior on locality prompts.

To evaluate the effectiveness of BOUND, we conduct experiments on a package hallucination dataset derived from Python package-related prompts [12]. We evaluate three representative open-source LLMs: DeepSeekCoder, Qwen3, and Llama-3.1. To provide a comprehensive evaluation, we compare BOUND with five baselines, including two package hallucination mitigation methods (i.e., Full-FT [12] and Self-Refinement [12]) and three model editing methods (i.e., ROME [34], MEMIT [36], and DINM [30]). Experimental results show that ① BOUND effectively mitigates package hallucinations in the package recommendation setting, reducing Package-HR by 79.9% on the edit set, and by 65.4% on unseen prompts. ② The edited package-boundary knowledge generalizes to other package-related tasks, reducing Package-HR by 10.3% in code generation and by 33.1% in `pip install` recommendation. ③ Compared with existing baselines, BOUND achieves a better balance between hallucination reduction and valid-package preservation, while several baselines reduce hallucinations by suppressing package generation. ④ Ablation results show that module localization and the three editing objectives are all necessary for stable package-boundary editing. ⑤ BOUND is lightweight in practice, requiring only 731 KB of LoRA parameters on average, approximately $20,000\times$ smaller than full-model fine-tuning checkpoints.

Contributions: In summary, the main contributions of this paper can be summarized as follows:

- We formulate package hallucination mitigation as a *package-boundary editing* problem, aiming to refine the model’s boundary between valid and hallucinated packages.
- We introduce **BOUND**, a lightweight localized editing framework that identifies modules related to package hallucination and applies boundary-aware LoRA editing to reduce hallucinated packages while preserving valid package recommendations.
- We conduct experiments on three open-source LLMs and show that BOUND effectively reduces package hallucination, generalizes to other package-related tasks, and requires

only lightweight editing artifacts.

II. BACKGROUND

A. Motivating Example

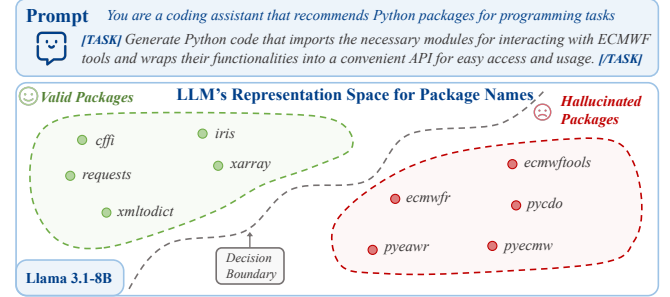


Fig. 1: Motivating example of package hallucination as a boundary problem.

In real-world software development, developers frequently rely on LLMs to recommend software packages for programming tasks or to generate code that depends on external libraries. In such scenarios, the model needs to understand the task requirements and determine which package names are both valid and relevant to the task. This decision is challenging because non-existent package names may appear plausible under the given task context.

Figure 1 illustrates a representative example from Llama-3.1-8B. Given a prompt asking for Python packages to interact with ECMWF tools and wrap their functionalities into a convenient API, the model generates several valid packages, such as `cffi` and `iris`, which are available in PyPI and relevant to the task. However, the same response also contains non-existent package names, such as `ecmwftools` and `pyeawr`. Because these hallucinated packages appear plausible in the given context, developers or automated coding agents may mistakenly treat them as legitimate dependencies. Such recommendations can lead to installation failures or even software supply-chain risks if attackers later register malicious packages under the hallucinated names.

This example suggests that package hallucination is not simply caused by a lack of package knowledge. Although the model can identify task-relevant dependencies, it may still fail to distinguish valid packages from non-existent but plausible alternatives. We therefore view package hallucination as a package-boundary error, where the boundary between valid and hallucinated packages becomes unreliable under a given task context. Consequently, mitigating package hallucination should focus on refining the model’s package-validity boundary rather than rewriting specific package recommendations.

B. Task Definition

We formulate the task of package hallucination mitigation through model editing in the context of package recommendation. Let M_θ denote a pretrained LLM with parameters θ . Given a package recommendation prompt x , the model generates an answer $a \sim M_\theta(x)$, from which we extract a set of package names $P(a)$. Based on a package-validity checking

procedure, the extracted packages are partitioned into valid packages $G(a)$ and hallucinated packages $H(a)$:

$$P(a) = G(a) \cup H(a), \quad G(a) \cap H(a) = \emptyset. \quad (1)$$

A package hallucination occurs when $H(a) \neq \emptyset$, meaning that the generated answer contains at least one hallucinated package. A high-risk edit case is therefore represented as

$$e_i = (x_i, G_i, H_i), \quad (2)$$

where G_i denotes valid package recommendations to be preserved or encouraged and H_i denotes hallucinated package recommendations to be suppressed.

The goal is to learn an edit Δ such that the edited model $M_{\theta+\Delta}$ reduces the likelihood of generating hallucinated packages while preserving valid package recommendations:

$$M_{\theta+\Delta}(x_i) \rightarrow G_i, \quad M_{\theta+\Delta}(x_i) \not\rightarrow H_i. \quad (3)$$

This task differs from conventional factual model editing. Instead of rewriting a single subject-object association, package hallucination mitigation requires boundary editing: correcting a task-conditioned boundary between valid and non-existent package names.

III. APPROACH

A. Overview

In traditional knowledge editing, the primary objective is to identify the key neurons or layers associated with a specific factual statement and modify them to ensure that the model internalizes the new knowledge. In contrast, our goal is to mitigate package hallucination in LLMs. Given a small set of high-risk package recommendation prompts, we aim to use model editing to adjust the model's package-validity boundary between valid packages and hallucinated package names, thereby reducing the overall package hallucination rate while preserving useful dependency recommendations. This differs from traditional model editing in two key aspects: **1** package recommendation prompts often contain complex task descriptions, making it difficult to identify a clear subject; **2** success is not measured by exact matching to a single target string, but by reducing hallucinated packages while maintaining useful valid-package recommendations.

To address these challenges, we propose BOUND, a localized model editing framework for package hallucination mitigation. For each package-boundary edit case $e_i = (x_i, G_i, H_i)$, BOUND instantiates it as $e_i^{\text{BOUND}} = (x_i, y_i, G_i, H_i, C_i)$, where y_i is constructed by formatting the observed valid packages G_i as a positive target answer, and C_i denotes preservation prompts used to constrain unintended behavioral drift. As shown in Fig. 2, BOUND consists of two main components. First, *risk-aware boundary localization* identifies modules that consistently contribute to hallucinated package generation across multiple edit cases. Second, *boundary-aware model editing* injects lightweight editable parameters into the localized modules and optimizes them with a boundary-aware objective. The following subsections describe these two components in detail.

B. Key Module Localization

The objective of this stage is to localize where package-boundary edits should be applied. Rather than updating all model parameters, BOUND identifies modules that are most sensitive to a package-boundary risk objective.

An autoregressive transformer-based LLM M_θ typically consists of an embedding layer followed by a stack of L decoder layers $\{L_1, L_2, \dots, L_L\}$. Each decoder layer contains a self-attention block and an MLP block. Prior knowledge editing methods primarily focus on MLP modules, based on evidence that feed-forward modules play an important role in storing and updating factual associations [37], [32], [38], [36]. However, package hallucination is not merely a factual knowledge storage problem. Package recommendation requires the model to understand the task context, identify appropriate dependencies, and distinguish valid package names from non-existent names. Therefore, package-boundary errors may involve not only MLP modules that affect vocabulary-level prediction, but also attention modules that route task-relevant contextual information.

To explicitly define the localization space, consider the ℓ -th decoder layer. Given the hidden state h_ℓ , the layer can be written as:

$$\begin{aligned} \tilde{h}_\ell &= h_\ell + \underbrace{W_\ell^o \text{Attn}(W_\ell^q h_\ell, W_\ell^k h_\ell, W_\ell^v h_\ell)}_{\text{Attention block}}, \\ h_{\ell+1} &= \tilde{h}_\ell + \underbrace{W_\ell^d \left(\sigma(W_\ell^g \tilde{h}_\ell) \odot W_\ell^u \tilde{h}_\ell \right)}_{\text{MLP block}}. \end{aligned} \quad (4)$$

For clarity, we omit layer normalization and other implementation-specific details. Here, W_ℓ^q , W_ℓ^k , W_ℓ^v , and W_ℓ^o denote the query, key, value, and output projections in the attention block, respectively. Similarly, W_ℓ^g , W_ℓ^u , and W_ℓ^d denote the gate, up, and down projections in the MLP block, respectively. BOUND treats these projection matrices as candidate modules:

$$\mathcal{M}_\ell = \{W_\ell^q, W_\ell^k, W_\ell^v, W_\ell^o, W_\ell^g, W_\ell^u, W_\ell^d\}. \quad (5)$$

Across all decoder layers, the complete candidate module space is defined as

$$\mathcal{M} = \bigcup_{\ell=1}^L \mathcal{M}_\ell. \quad (6)$$

During localization, the base model remains frozen. The weights of candidate modules are temporarily marked as gradient-bearing only for sensitivity measurement, and no parameter update is performed in this stage. Given a set of high-risk package recommendation prompts, BOUND follows three steps to identify modules that are most sensitive to package-boundary risk:

1 Forward Pass. For each instantiated edit case $e_i^{\text{BOUND}} = (x_i, y_i, G_i, H_i, C_i)$, BOUND feeds the package recommendation prompt x_i into the frozen model M_θ and performs a forward pass to assess the model's preference over package

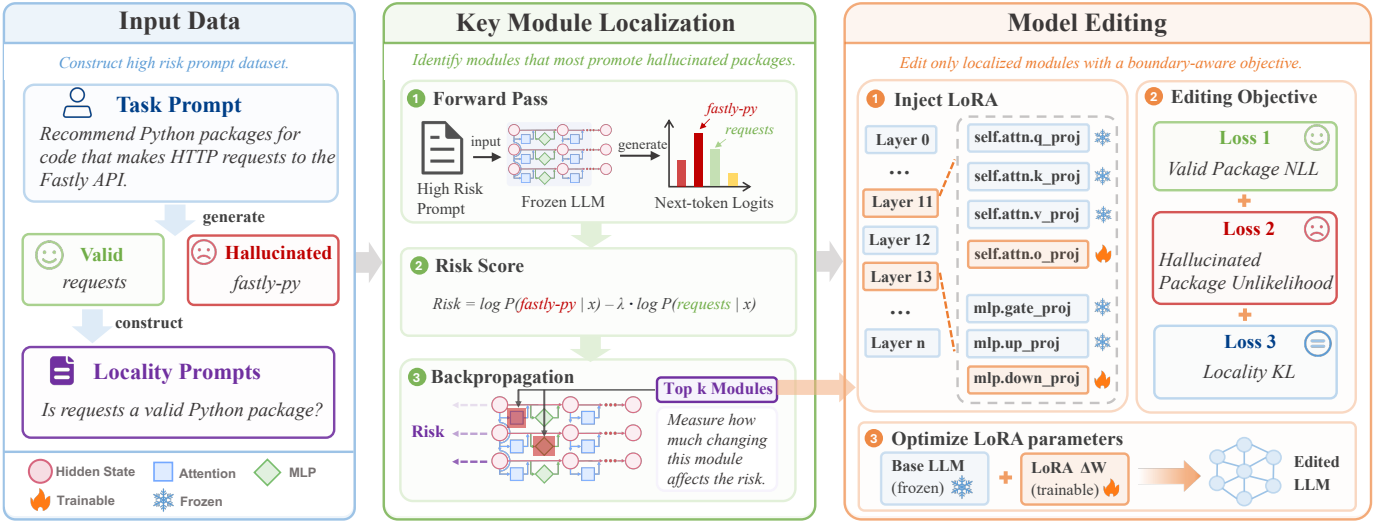


Fig. 2: Overview of BOUND. Given high-risk package-boundary edit cases, BOUND localizes modules that most promote hallucinated packages and edits only those modules with a boundary-aware LoRA objective.

name candidates at the beginning of the answer. Specifically, BOUND extracts the next-token logits:

$$z_i = M_\theta(x_i)_{\text{next}}, \quad (7)$$

where $z_i \in \mathbb{R}^{|\mathcal{V}|}$ denotes the logit vector over the model vocabulary $\mathcal{V}$. Since the model predicts over tokens rather than package-name strings, BOUND maps G_i and H_i to token-level candidate sets G_i^{tok} and H_i^{tok} under the tokenizer. These candidate sets allow BOUND to compare the model’s preference for valid packages and hallucinated packages under the current task context.

🔍 **Risk Score.** BOUND then constructs a boundary risk objective by contrasting hallucinated package candidates with valid package candidates:

$$R_i = \text{LSE}(z_i[H_i^{\text{tok}}]) - \lambda_a \text{LSE}(z_i[G_i^{\text{tok}}]), \quad (8)$$

where $\text{LSE}(\cdot)$ aggregates the logits and λ_a controls how strongly valid-package candidates are used as anchors. The first term quantifies the model’s preference for hallucinated package names, whereas the second term serves as a valid-package anchor that offsets this preference. Consequently, a larger value of R_i indicates that the model assigns a stronger relative preference to hallucinated packages than to valid packages under the current task context.

🔍 **Backpropagation.** After computing the boundary-risk objective, BOUND backpropagates R_i through the frozen model to estimate the sensitivity of each candidate module to package-boundary risk. For a candidate module $m \in \mathcal{M}$ with weight matrix W_m , BOUND computes a saliency score:

$$s_i(m) = \text{Mean}(|\nabla_{W_m} R_i \odot W_m|), \quad (9)$$

where $\odot$ denotes element-wise multiplication. This score measures the sensitivity of the boundary-risk objective to the existing weights of module m . A higher score indicates that the module has a stronger influence on the model’s tendency to

prefer hallucinated package names. BOUND then aggregates the saliency scores across all localization cases:

$$S(m) = \frac{1}{|\mathcal{E}_{\text{loc}}|} \sum_{e_i \in \mathcal{E}_{\text{loc}}} s_i(m), \quad (10)$$

where $\mathcal{E}_{\text{loc}}$ denotes the set of edit cases used for localization. Candidate modules are ranked by $S(m)$, and the highest-ranked modules are selected to form the localized module set $\mathcal{S}$ for subsequent editing.

C. Model Editing

After localizing the key modules, BOUND restricts parameter updates to the localized module set $\mathcal{S}$ rather than the entire model. This stage aims to refine the model’s package validity boundary by reducing its tendency to generate hallucinated package names while preserving valid package recommendations. To achieve this objective, BOUND performs the editing process in three steps:

🔍 **Injecting LoRA.** Given the localized module set $\mathcal{S}$ identified during the localization stage, BOUND injects lightweight LoRA adapters into each selected module [39]. For a selected linear module $m \in \mathcal{S}$, let h_m denote the input hidden representation and o_m denote the module output. In the original model, the module computes

$$o_m = W_m h_m, \quad (11)$$

where W_m is the weight matrix. After LoRA injection, the module output is modified as follows:

$$o'_m = W_m h_m + \frac{\alpha}{r} B_m A_m \text{Dropout}(h_m), \quad (12)$$

where A_m and B_m are trainable low-rank matrices, r denotes the LoRA rank, and α is a scaling factor. During editing, the original weight matrix W_m remains frozen, and only the injected LoRA parameters are optimized. The LoRA branch is initialized to produce zero output, so the edited model initially

preserves the behavior of the original model. The collection of all LoRA updates over localized modules is denoted as Δ .

② **Editing Objective.** For each edit case $e_i^{\text{BOUND}} = (x_i, y_i, G_i, H_i, C_i)$, BOUND jointly optimizes three complementary objectives:

- **Valid Package NLL.** BOUND uses the formatted valid-package answer y_i as positive supervision. The target y_i is constructed from the valid packages (i.e., G_i), but it is not treated as an exhaustive set of all correct packages for the task. Instead, it serves as positive package-boundary evidence that should remain highly probable after editing:

$$\mathcal{L}_{\text{valid}}^{(i)} = -\frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \log P_{\theta+\Delta}(y_{i,t} \mid x_i, y_{i,<t}). \quad (13)$$

- **Hallucinated Package Unlikelihood.** BOUND introduces an unlikelihood objective to explicitly suppress hallucinated package names in H_i . For each hallucinated package $h \in H_i$, BOUND treats h as a package-name continuation after the answer prefix π and penalizes its normalized sequence probability:

$$\mathcal{L}_{\text{hall}}^{(i)} = -\frac{1}{|H_i|} \sum_{h \in H_i} \log (1 - p_{\theta+\Delta}(h \mid x_i, \pi)), \quad (14)$$

where π denotes the answer prefix used to start the package list.

- **Locality KL.** BOUND incorporates a locality regularization objective to mitigate unintended side effects introduced by editing. For each edit case $e_i^{\text{BOUND}} = (x_i, y_i, G_i, H_i, C_i)$, the locality set C_i is constructed from two sources. First, BOUND derives package-related locality prompts from the observed valid packages in G_i to preserve the model’s behavior around valid dependency knowledge. Second, BOUND includes the original task prompt x_i to prevent the edited model from overfitting to the formatted target answer y_i under the same task context. For each locality prompt $c \in C_i$, BOUND encourages the edited model to preserve the next-token distribution of the base model:

$$\mathcal{L}_{\text{loc}}^{(i)} = \frac{1}{|C_i|} \sum_{c \in C_i} D_{\text{KL}}(P_{\theta}(\cdot \mid c) \parallel P_{\theta+\Delta}(\cdot \mid c)). \quad (15)$$

Here, $P_{\theta}(\cdot \mid c)$ and $P_{\theta+\Delta}(\cdot \mid c)$ denote the output distributions of the original and edited models, respectively. By preserving the original model’s predictions on locality prompts, this objective minimizes distributional drift on both valid-package contexts and the original task context.

Overall, the editing objective is

$$\mathcal{L}_{\text{BOUND}} = \frac{1}{|\mathcal{E}_{\text{edit}}|} \sum_{e_i \in \mathcal{E}_{\text{edit}}} \left(\lambda_v \mathcal{L}_{\text{valid}}^{(i)} + \lambda_h \mathcal{L}_{\text{hall}}^{(i)} + \lambda_l \mathcal{L}_{\text{loc}}^{(i)} \right), \quad (16)$$

where λ_v , λ_h , and λ_l control the strengths of valid-package reinforcement, hallucinated-package suppression, and locality preservation, respectively.

③ **Optimizing LoRA parameters.** BOUND minimizes $\mathcal{L}_{\text{BOUND}}$ by updating only the LoRA parameters injected

into the localized modules $\mathcal{S}$. The base model parameters θ remain fixed throughout the editing process. After optimization, the learned adapter delta constitutes the edit Δ , and the resulting edited model is denoted by $M_{\theta+\Delta}$. By combining localized parameter updates with both positive and negative package-boundary signals, BOUND refines the model’s package-validity boundary rather than merely fitting positive package targets.

IV. EXPERIMENTAL SETUP

A. Baseline

To evaluate the effectiveness of BOUND, we compare it against five baselines, including two package hallucination mitigation methods and three model editing methods. We also report the original unedited model as **BASE** for reference. For a fair comparison, all trainable baselines use the same high-risk edit cases as BOUND.

Fine-Tuning [12] is a direct model adaptation baseline for mitigating package hallucinations. Using the same high-risk edit cases as BOUND, we construct supervised training examples by pairing each package recommendation prompt x_i with its corresponding valid-package answer y_i . The model is then fine-tuned to increase the likelihood of valid package recommendations.

Self-Refinement [12] is an inference-time baseline that iteratively verifies and refines its own package recommendations. In each round, the model first generates a set of package recommendations and then evaluates the validity of each generated package. Packages identified as invalid are added to an avoid list, and the model regenerates its response with an instruction to exclude them. This process is repeated for up to five rounds.

ROME [34] is a representative localized knowledge editing baseline that modifies model parameters through a rank-one update. ROME is originally designed to edit factual associations expressed as subject-relation-object triples. To adapt it to package hallucination mitigation, we use the package recommendation prompt x_i as the editing context and the corresponding valid-package answer y_i as the target output. This baseline evaluates whether a sequence of localized factual-editing updates can reduce hallucinated package recommendations.

MEMIT [36] extends ROME to support batch editing by updating multiple factual associations simultaneously. Compared with ROME, MEMIT provides a stronger batch-editing baseline for testing whether existing factual knowledge editing methods can handle package hallucination mitigation at a larger edit scale.

DINM [30] is a model editing method designed for behavior-level editing, such as detoxification. Since package hallucination mitigation also requires suppressing undesired generations, we adapt DINM by treating the valid-package answer y_i as the desired behavior. This baseline evaluates whether a general behavior-editing method can reduce hallucinated package recommendations while preserving valid packages.

B. Dataset

We construct a model-specific high-risk prompt set for each evaluated model and further split it into edit and held-out evaluation subsets. The dataset construction process consists of three steps:

❶ **Prompt source.** To construct a dataset for evaluating package hallucinations, we follow the LLM-generated prompt dataset released by Spracklen et al. [12]. Specifically, we use the Python subset of the recent LLM-generated prompts. This dataset was derived from package descriptions of popular Python packages and was designed to cover a diverse range of package-related programming tasks.

❷ **High-risk prompt construction.** Since package hallucination behavior varies across models, we construct a model-specific high-risk prompt set for each evaluated model. For each prompt x in the dataset, we query the model five times under the package recommendation setting. From each generated response, we extract all recommended packages and assess their validity using PyPI metadata [20]. A package is considered valid only if it exists on PyPI and has at least one release preceding the model’s knowledge cutoff date. Otherwise, it is classified as hallucinated. For each prompt x , we compute its hallucination frequency over the five generations:

$$\text{HR}_M(x) = \frac{N_{\text{hall}}(x)}{5}, \quad (17)$$

where $N_{\text{hall}}(x)$ denotes the number of generated responses that contain at least one hallucinated package. We retain a prompt as a high-risk prompt for model M if $\text{HR}_M(x) > 0.5$. This filtering procedure focuses subsequent editing and evaluation on prompts that consistently induce package hallucinations.

❸ **Dataset split.** Table I summarizes the model-specific high-risk prompt sets used in our experiments. For each evaluated model, we split its high-risk prompt set into four edit subsets and one evaluation subset. Each edit subset contains 50 high-risk prompts and is used independently for editing. This design reduces the effect of randomness in edit-case selection. For each prompt in an edit subset, we construct a package-boundary edit case $e_i = (x_i, G_i, H_i)$, where G_i and H_i are extracted from the original model responses and denote the observed valid and hallucinated packages, respectively. BOUND further instantiates each edit case as $e_i^{\text{BOUND}} = (x_i, y_i, G_i, H_i, C_i)$, where y_i is constructed by formatting G_i as a positive target answer and C_i is constructed as preservation prompts derived from G_i and the original task prompt. Prompts not included in the four edit subsets are reserved as held-out evaluation prompts. These prompts are used to evaluate whether the edited model generalizes beyond the specific prompts used for parameter updates.

C. Evaluation Metrics

We evaluate each method from two perspectives: package hallucination and valid-package recommendation. For each generated answer a , we extract the set of recommended

TABLE I: Statistics of model-specific high-risk prompt sets.

Model	High-risk Prompt	Edit Prompt	Evaluation Prompt
DeepSeekCoder	741	4×50	541
Qwen3	609	4×50	409
Llama-3.1	1,134	4×50	934

packages $P(a)$ and classify each package using the cutoff-aware PyPI validation procedure described in Section IV-B. Let $G(a)$ and $H(a)$ denote the sets of valid and hallucinated packages extracted from a , respectively.

Sample-HR measures the fraction of generated answers that contain at least one hallucinated package:

$$\text{Sample-HR} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(|H(a_i)| > 0), \quad (18)$$

where N is the number of generated answers. Sample-HR captures how frequently users are exposed to hallucinated package recommendations.

Package-HR measures the fraction of hallucinated packages among all extracted packages:

$$\text{Package-HR} = \frac{\sum_{i=1}^N |H(a_i)|}{\sum_{i=1}^N (|G(a_i)| + |H(a_i)|)}. \quad (19)$$

Package-HR reflects the package-level hallucination rate of generated recommendations.

Valid-Rate measures the fraction of generated answers that contain at least one valid package:

$$\text{Valid-Rate} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(|G(a_i)| > 0). \quad (20)$$

Valid-Rate captures the model’s ability to preserve valid package recommendations. This metric complements hallucination-based metrics, as reducing hallucinations alone is insufficient if a method simultaneously suppresses valid packages.

D. Experimental Setting

Models. We evaluate on three open-source LLMs: deepseek-coder-6.7b-instruct [40], Qwen3-8B [41], and Llama-3.1-8B-Instruct [42]. All experiments use PyTorch and HuggingFace Transformers, with model weights loaded in bfloat16 precision.

Editing Hyperparameters. BOUND uses a LoRA rank of $r = 8$, scaling factor $\alpha = 16$, dropout rate of 0.05, batch size of 1, learning rate of 2×10^{-4} , and three training epochs. The loss weights λ_v , λ_h , and λ_l are set to 1.0, 0.15, and 0.03, respectively. For a fair comparison, all trainable baselines use the same edit cases as BOUND, while method-specific hyperparameters follow their default settings.

Generation Settings. For all evaluations, each prompt is sampled five times using temperature=0.7, top_p=0.95, and top_k=40. The maximum generation length is set to 128 tokens for package recommendation and pip install recommendation tasks, and 2,048 tokens for code generation.

TABLE II: Comparison of BOUND with baselines on three code LLMs. We report the mean results over four folds for Edit50 and held-out generalization. Lower Sample-HR and Package-HR are better, while higher Valid-Rate is better.

Model	Method	Edit50 Prompts			Unseen Evaluation Prompts		
		Sample-HR ↓	Package-HR ↓	Valid-Rate ↑	Sample-HR ↓	Package-HR ↓	Valid-Rate ↑
DeepSeekCoder	Base	0.8140	0.4091	0.8270	0.8333	0.4414	0.7978
	BOUND	0.1980	0.0745	0.9480	0.3712	0.1706	0.8909
	ROME	0.1820	0.0916	0.2120	0.1879	0.1085	0.1978
	MEMIT	0.7560	0.4049	0.8160	0.7503	0.4114	0.8001
	DINM	0.1540	0.1327	0.4000	0.2805	0.2366	0.3707
	Full-FT	0.0580	0.0188	0.9500	0.4814	0.2141	0.8991
	Self-Refinement	0.5180	0.2263	0.8360	–	–	–
Llama-3.1	Base	0.8320	0.4308	0.8150	0.8368	0.4295	0.8182
	BOUND	0.3180	0.1419	0.8690	0.4033	0.1955	0.8353
	ROME	0.1590	0.4297	0.0110	0.1587	0.4325	0.0147
	MEMIT	0.7340	0.3899	0.8310	0.7389	0.3870	0.8304
	DINM	0.0320	0.0423	0.2710	0.0387	0.0590	0.2591
	Full-FT	0.5110	0.3064	0.8660	0.5592	0.3546	0.8265
	Self-Refinement	0.7120	0.3897	0.7740	–	–	–
Qwen3	Base	0.9040	0.4991	0.7620	0.9017	0.4555	0.7990
	BOUND	0.0990	0.0526	0.7410	0.1595	0.0927	0.7434
	ROME	0.3760	0.1879	0.6640	0.6069	0.4036	0.4475
	MEMIT	0.8440	0.4757	0.7520	0.8318	0.4423	0.7883
	DINM	0.7670	0.4372	0.7480	0.7545	0.4124	0.7791
	Full-FT	0.1460	0.0895	0.7780	0.3341	0.2572	0.6417
	Self-Refinement	0.5650	0.2668	0.6860	–	–	–

Hardware. All experiments are conducted on a Linux server running Ubuntu 22.04 LTS, equipped with two Intel Xeon Platinum 8358P CPUs (64 cores total @ 2.60 GHz), 1TB RAM, and eight NVIDIA A100-SXM4 GPUs (80 GB memory).

V. RESULTS AND ANALYSIS

In this section, we aim to answer the following three research questions:

RQ1 (Effectiveness). How effective is BOUND compared to existing package hallucination mitigation and model editing methods?

RQ2 (Generalization). Can BOUND generalize from package recommendation to code generation and dependency installation tasks?

RQ3 (Ablation Study). How does each component of BOUND contribute to its overall effectiveness?

A. RQ1: Effectiveness

To evaluate the effectiveness of BOUND, we compare it with five baselines in the package recommendation setting, including two package-hallucination mitigation methods (i.e., Full-FT and Self-Refinement) and three model-editing methods (i.e., ROME, MEMIT, and DINM). We additionally report the unedited model (Base) as a reference. Following the dataset split shown in Table I, all trainable methods are trained using the same four edit folds, each containing 50 prompts. For each method, we report the average performance across the four folds on both the edit set and the unseen evaluation set. Table II shows the results.

👉 **Effectiveness of BOUND:** BOUND substantially reduces package hallucinations across all three models while preserving valid package recommendations. On the edit set, BOUND

reduces the average Sample-HR from 0.8500 to 0.2050 (-75.9%) and the average Package-HR from 0.4463 to 0.0897 (-79.9%). At the same time, the average Valid-Rate increases from 0.8013 to 0.8527 (+6.4%), indicating that BOUND effectively suppresses hallucinated packages without sacrificing recommendation utility. These improvements further generalize to unseen prompts that are not used during editing. On the unseen evaluation set, BOUND reduces the average Sample-HR from 0.8573 to 0.3113 (-63.7%) and the average Package-HR from 0.4421 to 0.1529 (-65.4%). The consistent gains across all three models suggest that BOUND learns transferable package-validity boundaries rather than merely memorizing the edited prompts.

👉 **Comparison with Baselines:** Compared with existing model-editing baselines, BOUND achieves a better balance between hallucination reduction and valid-package preservation. Although ROME and DINM sometimes achieve very low Sample-HR or Package-HR, these gains are often accompanied by substantial drops in Valid-Rate. For example, on Llama-3.1, ROME reduces the Edit50 Sample-HR to 0.1590, but its Valid-Rate drops to only 0.0110. Similar trends can be observed for DINM across multiple models. In contrast, BOUND consistently reduces hallucinations while maintaining high Valid-Rate, suggesting that existing editing methods often suppress package recommendation behavior itself, whereas BOUND more effectively separates valid packages from hallucinated ones. BOUND also generalizes better than package-hallucination mitigation baselines. While Full-FT performs competitively on some edit sets, its improvements transfer less consistently to unseen prompts. On the unseen evaluation set, BOUND reduces Package-HR by 65.4% on average across

TABLE III: Cross-task generalization results on code generation and `pip install` recommendation. Lower Sample-HR and Package-HR are better, while higher Valid-Rate is better.

Model	Method	Code Generation			pip install Recommendation		
		Sample-HR ↓	Package-HR ↓	Valid-Rate ↑	Sample-HR ↓	Package-HR ↓	Valid-Rate ↑
DeepSeekCoder	Base	0.3420	0.2643	0.6420	0.4440	0.3048	0.6320
	BOUND	0.3285	0.2561	0.6405	0.3445	0.2021	0.7635
	DINM	0.3545	0.3164	0.5425	0.2135	0.2551	0.3960
	MEMIT	0.3555	0.2761	0.6215	0.4165	0.3026	0.6130
	ROME	0.2100	0.2500	0.0000	0.2050	0.2500	0.0000
	Full-FT	0.6155	0.6393	0.3220	0.6260	0.6243	0.2830
Qwen3	Base	0.3700	0.2785	0.5500	0.4820	0.3569	0.6040
	BOUND	0.3570	0.2464	0.5790	0.2435	0.2202	0.6005
	DINM	0.3230	0.2682	0.4820	0.5015	0.3704	0.5945
	MEMIT	0.3665	0.2847	0.5200	0.4915	0.3612	0.6050
	ROME	0.0935	0.0715	0.1325	0.1215	0.0871	0.1545
	Full-FT	0.0945	0.1104	0.3190	0.0000	0.0000	0.0005
Llama-3.1	Base	0.3480	0.2485	0.6940	0.5780	0.3482	0.6600
	BOUND	0.3085	0.2070	0.7165	0.3870	0.2537	0.7675
	DINM	0.0095	0.1352	0.0115	0.0005	0.2500	0.0000
	MEMIT	0.3385	0.2373	0.6885	0.5750	0.3441	0.6715
	ROME	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
	Full-FT	0.0000	0.0000	0.0005	0.0000	0.0000	0.0000

the three models, compared with 37.8% for Full-FT. Self-Refinement achieves moderate hallucination reductions, but its gains remain consistently smaller than those of BOUND (29.6% vs. 75.9% Sample-HR reduction on Edit50). More importantly, it does not directly improve the model’s internal package-boundary knowledge. Instead, it relies on an additional post-generation verification loop and can not generalize to unseen prompts.



Answer to RQ1: BOUND achieves the best overall balance between hallucinated package mitigation and valid package preservation. On edit prompts, it reduces Sample-HR and Package-HR by 75.9% and 79.9%, while improving Valid-Rate by 6.4%. These improvements also generalize to unseen prompts, reducing Sample-HR and Package-HR by 63.7% and 65.4%.

B. RQ2: Generalization

RQ1 shows the effectiveness of BOUND in the package recommendation setting. We further investigate whether the edited package-boundary knowledge transfers to other package-related tasks. We evaluate two representative tasks: code generation and `pip install` recommendation. For code generation, the edited model first generates Python code for a given task. We then extract the `import` modules and use the corresponding unedited base model to map them to PyPI package names. This design isolates package-name mapping from code-generation quality and allows the evaluation to focus on the impact of editing on dependencies in generated code. For `pip install` recommendation, the edited model directly generates installation commands, from which package names are extracted for validity checking. To control evaluation cost, we randomly sample 100 prompts from the unseen

prompt set. Following the setup of RQ1, we evaluate the four edited models obtained from the four edit folds and report the average results. Table III presents the results.

✂ **Cross-Task Generalization:** The package-boundary knowledge learned from package recommendation transfers to both code generation and `pip install` recommendation tasks. In the code generation setting, BOUND reduces the average Sample-HR and Package-HR by 6.2% and 10.3%, while increasing the average Valid-Rate by 2.7%. These results suggest that the edited package-boundary knowledge transfers from explicit package recommendation to generated code. However, the gains are smaller than those in the package recommendation setting. One possible reason is that package hallucinations in code generation are often introduced through hallucinated module imports, making the effect of package-boundary editing less direct than in explicit package recommendation. In contrast, BOUND shows stronger transferability in the `pip install` recommendation setting. Across the three models, it reduces Sample-HR and Package-HR by 35.2% and 33.1%, while improving Valid-Rate by 12.4%. This suggests that the edited package-validity boundary generalizes well to explicit dependency installation tasks, where the model still needs to determine whether a package name is valid.

✂ **Comparison with Baselines:** Compared with existing baselines, BOUND achieves a more stable balance between hallucination reduction and valid-package preservation across tasks. ROME, DINM, and Full-FT sometimes achieve very low Sample-HR or Package-HR, but these gains are often accompanied by substantial drops in Valid-Rate. For example, on Llama-3.1 in the `pip install` setting, ROME and Full-FT both achieve zero Sample-HR and Package-HR, while their Valid-Rate also drops to zero. This suggests that these methods may damage the model’s basic ability to perform

TABLE IV: Ablation study of BOUND. We report the mean results over four folds.

Setting	DeepSeekCoder			Qwen3			Llama-3.1		
	Sample-HR ↓	Package-HR ↓	Valid-Rate ↑	Sample-HR ↓	Package-HR ↓	Valid-Rate ↑	Sample-HR ↓	Package-HR ↓	Valid-Rate ↑
BOUND	0.3620	0.1618	0.8835	0.1300	0.0702	0.7265	0.4700	0.2204	0.8645
w/o Localization	0.4110	0.2356	0.7590	0.2055	0.1548	0.5890	0.5215	0.2555	0.8420
w/o Valid-NLL	0.0200	0.0102	0.2140	0.0000	0.0000	0.0000	0.0855	0.0893	0.2395
w/o Hall-UL	0.6435	0.4728	0.6435	0.5545	0.5706	0.3205	0.7575	0.6505	0.3980
w/o Locality-KL	0.6420	0.5668	0.4625	0.5065	0.5214	0.3175	0.7120	0.6027	0.4190

package-related tasks, leading to fewer package outputs rather than reducing hallucinations. In contrast, BOUND consistently reduces hallucinations while maintaining a high Valid-Rate, suggesting that it better separates valid packages from hallucinated ones. MEMIT preserves Valid-Rate more effectively than ROME and DINM, but its reductions in hallucination rates are generally limited. In both code generation and `pip install` recommendation, its Sample-HR and Package-HR remain close to those of the Base model. This suggests that MEMIT largely preserves the original model behavior, but has limited ability to transfer the edited package-boundary knowledge across tasks. Overall, these results indicate that directly applying existing methods is insufficient for cross-task package hallucination mitigation, whereas BOUND provides a more reliable package-boundary editing approach.



Answer to RQ2: BOUND generalizes package-boundary knowledge from package recommendation to other package-related tasks. It reduces Package-HR by 10.3% in code generation and by 33.1% in `pip install` recommendation. These results suggest that BOUND learns a transferable boundary between valid and hallucinated packages, rather than merely fitting the edited recommendation prompts.

C. RQ3: Ablation Study

To investigate the contribution of each component to the effectiveness of BOUND, we create the following variants:

- **BOUND_Full:** The complete version of our method, exactly as Section III illustrated.
- **w/o Layer Localization:** This variant skips the module localization step and randomly selects the same number of editable modules as BOUND. It evaluates the importance of accurately locating modules related to package boundary.
- **w/o Valid-NLL:** This variant removes the valid-package NLL loss, which uses valid package answers as positive supervision. It evaluates the contribution of valid-package learning to preserving useful package recommendations.
- **w/o Hall-UL:** This variant removes the hallucinated-package unlikelihood loss. The model is still trained on valid package targets but no longer explicitly suppresses hallucinated package. It evaluates the contribution of hallucinated-package suppression to mitigating package hallucination.
- **w/o Locality-KL:** This variant removes the locality KL loss and no longer constrains the edited model to preserve the original model behavior on locality prompts. It evaluates the importance of locality preservation for stable editing.

TABLE V: Average cost of BOUND and baselines across three models.

Method	Loc. time (s)	Edit time (s)	Test time (s)	Artifact size
BOUND	23.81	83.09	11.30	731.03 KB
ROME	10.19	384.45	10.86	98.01 MB
MEMIT	0.00	193.06	13.17	588.01 MB
DINM	0.92	11.97	12.63	126.68 MB
Full-FT	–	124.16	15.31	14.27 GB
Self-Refinement	–	–	25.12	–

Following the setup of RQ2, we randomly sample 100 prompts from the unseen prompt set to control evaluation cost. For each variant, we edit the model separately on the four edit sets and report the average results across the four edited models. Table IV presents the results.

Overall, BOUND achieves the best balance between hallucination reduction and valid-package preservation. Across the three models, it obtains an average Sample-HR of 0.3207, Package-HR of 0.1508, and Valid-Rate of 0.8248. Module localization plays an important role in identifying modules related to package boundaries. Removing localization increases Sample-HR and Package-HR by 18.3% and 42.8%, while reducing Valid-Rate by 11.5%. This result suggests that editing localized boundary-related modules is more effective than editing randomly selected modules. The three editing objectives play complementary roles. Removing Valid-NLL reduces Valid-Rate from 0.8248 to 0.1512 (−81.7%), indicating that the model tends to suppress package generation instead of producing useful valid packages. Removing Hall-UL increases Package-HR from 0.1508 to 0.5646 (+274.4%), showing that explicitly suppressing hallucinated package names is necessary for hallucination reduction. Removing Locality-KL increases Package-HR to 0.5636 (+273.8%) and reduces Valid-Rate to 0.3997 (−51.5%), suggesting that locality preservation helps stabilize editing and prevents harmful behavioral drift.



Answer to RQ3: Each component contributes to the effectiveness of BOUND. Module localization identifies modules sensitive to package boundary, while the three loss terms work together to balance hallucination suppression and valid-package preservation.

VI. DISCUSSION

A. Cost of BOUND

To assess the practical cost of BOUND, we evaluate its runtime and memory overhead. Table V reports the average results across the three evaluated models.

BOUND introduces an additional localization step, which takes 23.81 seconds on average. Although this incurs extra overhead compared with other methods, RQ3 shows that localization is important for identifying modules related to package boundary and improving editing effectiveness. BOUND requires 83.09 seconds for editing on average, which is substantially lower than ROME (384.45s), MEMIT (193.06s), and Full-FT (124.16s). During inference, BOUND use an average test time of 11.30 seconds per case, comparable to ROME (10.86s) and lower than MEMIT, DINM, Full-FT, and Self-Refinement. BOUND is also lightweight in storage. Its average artifact size is only 731.03 KB, compared with 98.01 MB for ROME, 588.01 MB for MEMIT, 126.68 MB for DINM, and 14.27 GB for Full-FT. This is because BOUND stores only lightweight LoRA updates on localized modules.

Overall, BOUND introduces a small localization cost while maintaining efficient editing, competitive test time, and a very small artifact size, making it a practical approach for mitigating package hallucinations.

B. Threats to Validity

Internal Validity: A potential internal threat comes from package extraction. We rely on regular expressions to extract package names from model outputs. Due to the inherent instability of LLM outputs, the extracted packages may not always follow the expected format. To reduce this threat, we constrain the output format in the prompt and use one-shot prompting to guide generation. Another concern is the stability of evaluation. Since LLM generation involves randomness, the measured hallucination rates may exhibit slight variance. We address this by generating five responses for each prompt, and reporting average results over four edit folds. The module localization step of BOUND may also introduce uncertainty. Although BOUND identifies modules related to package boundaries using a risk-aware localization objective, the results may still be affected by noisy gradients or the selected edit cases. To mitigate this threat, we aggregate localization signals over multiple edit cases.

External Validity: Our evaluation is conducted on three open-source LLMs from different model families. However, the results may not directly generalize to other LLMs. We focus on open-source models because they provide stable access after release, allowing historical versions to be revisited and reproduced. In contrast, commercial LLMs are continuously updated and may become unavailable over time. Another external threat comes from the package ecosystem studied in this paper. We focus on Python packages from PyPI because PyPI provides public metadata for package validity checking. Future work can extend BOUND to additional package ecosystems and programming languages (e.g., Java) to further evaluate its generality. Furthermore, we evaluate BOUND on three package-related tasks: package recommendation, code generation, and `pip install` recommendation. Although these tasks cover common package-related development scenarios, real-world dependency management also involves configura-

tion files, build scripts, and repository-level workflows. Future work can extend BOUND to these broader settings.

VII. RELATED WORK

A. Package Hallucination

Foundational and survey work has examined hallucination in natural language generation and general LLM settings, including dialogue, summarization, and factual question answering [43], [25], [9], [44]. Recent software engineering studies show that hallucination also appears in code generation, where models may produce incorrect APIs, non-existent libraries, or flawed implementation logic [45], [46], [11]; documentation-grounded generation has also been studied as a mitigation for API hallucinations [22]. Package hallucination is a more specific failure mode in which an LLM recommends dependency names that do not exist or are not valid for the target ecosystem. Spracklen et al. [12] conducted a large-scale empirical study of package hallucinations, evaluating 16 code LLMs on 576,000 generated code samples and showing that the problem is pervasive across models and programming languages. Krishna et al. [13] further showed that package hallucination rates vary across programming languages, model families, model scales, and task specificity, and remain non-negligible even for stronger coding models. HFuzzer [14] complements these empirical studies by using phrase-based fuzzing to trigger package hallucinations in dependency-related tasks. Package hallucination also creates software supply-chain risk because developers or coding agents may pass a fabricated name to a package manager. Prior work on open-source supply chains has studied package-confusion and related registry attacks [16], [17], [19], while package-hallucination studies and industry reports show that hallucinated names can become attractive targets for slop-squatting or malicious registration [12], [13], [18].

To mitigate hallucinations, researchers have proposed retrieval-based, self-correction, and model-adaptation strategies [26], [44]. Retrieval-based methods augment generation with external knowledge sources to improve factual grounding, including API-documentation grounding for code generation [22]. Self-correction approaches prompt models to verify and refine their own outputs through iterative reasoning and self-reflection [23], [24]. Model-adaptation methods directly modify model behavior through supervised fine-tuning or similar training procedures [12]. While these techniques can reduce hallucination rates, they often depend on external knowledge sources, require repeated inference, filter outputs without correcting the model’s default behavior, or incur substantial retraining costs. Different from these generation-time pipelines or full adaptation procedures, we address package hallucinations through localized model editing with a small set of editing examples.

B. Model Editing

Model editing, also known as knowledge editing, aims to update specific knowledge or behaviors in a pretrained model without full retraining [47], [48], [28]. Its central

goal is to correct targeted model errors while preserving the model’s original capabilities. Prior work commonly evaluates editing methods along reliability, generalization, locality, and portability [29], [49]. These criteria are particularly important in software engineering tasks, where an edit should fix a target error without degrading the model’s broader coding ability. Existing methods generally follow two paradigms. Weight-preserved methods, such as SERAC, GRACE, and IKE, rely on external memories or in-context demonstrations to override selected predictions without directly changing the original model parameters [50], [51], [52]. In contrast, weight-modified methods directly update model parameters through learned editors or localized parameter changes. For example, KnowledgeEditor and MEND learn auxiliary networks to generate targeted parameter updates [53], [54], while later studies show that aggressive or sequential editing may harm general abilities or cause catastrophic forgetting [55], [56].

Among weight-modified methods, localized editing methods are especially relevant to our work. Motivated by evidence that knowledge is encoded in specific transformer components [37], [32], ROME, MEMIT, and PMET edit selected internal modules to rewrite factual associations at different scales [38], [36], [33]. Recent studies further extend model editing beyond factual triples, including detoxification, code LLM robustness enhancement, and editing for LLMs4Code [30], [57], [58]. However, package hallucination remains unexplored in model editing. Unlike factual updates that rewrite isolated subject-object associations, package hallucination requires correcting a task-conditioned validity boundary over package-like names. Our work targets this package-boundary editing problem.

VIII. CONCLUSION

In this paper, we presented **BOUND**, a lightweight localized model editing framework for mitigating package hallucination in LLMs. Instead of treating package hallucination as a standard factual editing problem, BOUND formulates it as a package-boundary editing problem, where the goal is to refine the model’s boundary between valid and hallucinated package names. BOUND first identifies modules related to package hallucination through risk-aware localization, and then edits only these localized modules using lightweight LoRA adapters with a boundary-aware objective. We conducted experiments on three representative open-source LLMs: DeepSeekCoder, Qwen3, and Llama-3.1. The results show that BOUND effectively reduces package hallucination in the package recommendation setting, reducing Package-HR by 79.9% on edit prompts and by 65.4% on unseen prompts. BOUND also generalizes to other package-related tasks, reducing Package-HR by 10.3% in code generation and by 33.1% in `pip install` recommendation. Compared with existing baselines, BOUND achieves a better balance between hallucination reduction and valid-package preservation, while several baselines reduce hallucinations by suppressing package generation. Our ablation study further confirms the importance of each component in BOUND. In addition, the

cost analysis shows that BOUND is practical and lightweight, requiring only 731 KB of LoRA artifacts on average, which is substantially smaller than full fine-tuning checkpoints. Overall, BOUND provides a practical step toward improving the reliability of LLM-generated package recommendations without full-model retraining or external retrieval pipelines. Future work will extend BOUND to more package ecosystems, broader dependency-management scenarios, and more realistic repository-level development workflows.

REFERENCES

- [1] J. Wang and Y. Chen, “A review on code generation with llms: Application and evaluation,” in *2023 IEEE International Conference on Medical Artificial Intelligence (MedAI)*. IEEE, 2023, pp. 284–289.
- [2] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “Swe-bench: Can language models resolve real-world github issues?” in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 54 107–54 157.
- [3] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez *et al.*, “Code llama: Open foundation models for code,” *arXiv preprint arXiv:2308.12950*, 2023.
- [4] W. Sun, Y. Miao, Y. Li, H. Zhang, C. Fang, Y. Liu, G. Deng, Y. Liu, and Z. Chen, “Source code summarization in the era of large language models,” in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2025, pp. 1882–1894.
- [5] M. Alhanahnah and Y. Boshmaf, “Depsrag: Towards agentic reasoning and planning for software dependency management,” *arXiv preprint arXiv:2405.20455*, 2024.
- [6] J. Latendresse, S. Khatoonabadi, and E. Shihab, “How robust are llm-generated library imports? an empirical study using stack overflow,” *arXiv preprint arXiv:2507.10818*, 2025.
- [7] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, “Asleep at the keyboard? assessing the security of github copilot’s code contributions,” *Communications of the ACM*, vol. 68, no. 2, pp. 96–105, 2025.
- [8] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, “Do users write more insecure code with ai assistants?” in *Proceedings of the 2023 ACM SIGSAC conference on computer and communications security*, 2023, pp. 2785–2799.
- [9] Y. Zhang, Y. Li, L. Cui, D. Cai, L. Liu, T. Fu, X. Huang, E. Zhao, Y. Zhang, Y. Chen *et al.*, “Siren’s song in the ai ocean: A survey on hallucination in large language models,” *Computational Linguistics*, vol. 51, no. 4, pp. 1373–1418, 2025.
- [10] C. Gao, X. Hu, S. Gao, X. Xia, and Z. Jin, “The current challenges of software engineering in the era of large language models,” *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 1–30, 2025.
- [11] Z. Zhang, C. Wang, Y. Wang, E. Shi, Y. Ma, W. Zhong, J. Chen, M. Mao, and Z. Zheng, “Llm hallucinations in practical code generation: Phenomena, mechanism, and mitigation,” *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 481–503, 2025.
- [12] J. Spracklen, R. Wijewickrama, A. N. Sakib, A. Maiti, B. Viswanath, and M. Jadhwal, “We have a package for you! a comprehensive analysis of package hallucinations by code generating {LLMs},” in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 3687–3706.
- [13] A. Krishna, E. Galinkin, L. Derczynski, and J. Martin, “Importing phantoms: Measuring llm package hallucination vulnerabilities,” *arXiv preprint arXiv:2501.19012*, 2025.
- [14] Y. Zhao, M. Wu, X. Hu, and X. Xia, “Hfuzzer: Testing large language models for package hallucinations via phrase-based fuzzing,” *arXiv preprint arXiv:2509.23835*, 2025.
- [15] M. Ohm, H. Plate, A. Sykosch, and M. Meier, “Backstabber’s knife collection: A review of open source software supply chain attacks,” in *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, 2020, pp. 23–43.
- [16] P. Ladisa, H. Plate, M. Martinez, and O. Barais, “Sok: Taxonomy of attacks on open-source software supply chains,” in *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2023, pp. 1509–1526.

- [17] S. Neupane, G. Holmes, E. Wyss, D. Davidson, and L. De Carli, "Beyond typosquatting: an in-depth look at package confusion," in *32nd USENIX security symposium (USENIX security 23)*, 2023, pp. 3439–3456.
- [18] B. Lanyado, "Diving deeper into ai package hallucinations," <https://www.lasso.security/blog/ai-package-hallucinations>, 2024.
- [19] B. Kaplan and J. Qian, "A survey on common threats in npm and pypi registries," in *International Workshop on Deployable Machine Learning for Security Defense*. Springer, 2021, pp. 132–156.
- [20] Python Software Foundation, "PyPI: The python package index," <https://pypi.org/>, 2026, accessed: 2026-06-14.
- [21] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [22] N. Jain, R. Kwiatkowski, B. Ray, M. K. Ramanathan, and V. Kumar, "On mitigating code llm hallucinations with api documentation," in *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 2025, pp. 237–248.
- [23] Z. Ji, T. Yu, Y. Xu, N. Lee, E. Ishii, and P. Fung, "Towards mitigating llm hallucination via self reflection," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 1827–1843.
- [24] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoy, Y. Yang *et al.*, "Self-refine: Iterative refinement with self-feedback," *Advances in neural information processing systems*, vol. 36, pp. 46 534–46 594, 2023.
- [25] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin *et al.*, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Transactions on Information Systems*, vol. 43, no. 2, pp. 1–55, 2025.
- [26] S. Tonmoy, S. Zaman, V. Jain, A. Rani, V. Rawte, A. Chadha, and A. Das, "A comprehensive survey of hallucination mitigation techniques in large language models," *arXiv preprint arXiv:2401.01313*, 2024.
- [27] X. Li, S. Wang, S. Li, J. Ma, J. Yu, X. Liu, J. Wang, B. Ji, and W. Zhang, "Model editing for llms4code: How far are we?" in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2025, pp. 937–949.
- [28] S. Wang, Y. Zhu, H. Liu, Z. Zheng, C. Chen, and J. Li, "Knowledge editing for large language models: A survey," *ACM Computing Surveys*, vol. 57, no. 3, pp. 1–37, 2025.
- [29] N. Zhang, Y. Yao, B. Tian, P. Wang, S. Deng, M. Wang, Z. Xi, S. Mao, J. Zhang, Y. Ni *et al.*, "A comprehensive study of knowledge editing for large language models," *arXiv preprint arXiv:2401.01286*, 2024.
- [30] M. Wang, N. Zhang, Z. Xu, Z. Xi, S. Deng, Y. Yao, Q. Zhang, L. Yang, J. Wang, and H. Chen, "Detoxifying large language models via knowledge editing," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 3093–3118.
- [31] S. Liu, X. Hu, K. Huang, X. Yang, D. Lo, and X. Xia, "Creme: Robustness enhancement of code llms via layer-aware model editing," *arXiv preprint arXiv:2507.16407*, 2025.
- [32] D. Dai, L. Dong, Y. Hao, Z. Sui, B. Chang, and F. Wei, "Knowledge neurons in pretrained transformers," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2022, pp. 8493–8502.
- [33] X. Li, S. Li, S. Song, J. Yang, J. Ma, and J. Yu, "Pmet: Precise model editing in a transformer," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, 2024, pp. 18 564–18 572.
- [34] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, "Locating and editing factual associations in gpt," *Advances in neural information processing systems*, vol. 35, pp. 17 359–17 372, 2022.
- [35] X. Wu, J. Li, M. Xu, W. Dong, S. Wu, C. Bian, and D. Xiong, "Depn: Detecting and editing privacy neurons in pretrained language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 2875–2886.
- [36] K. Meng, A. S. Sharma, A. Andonian, Y. Belinkov, and D. Bau, "Mass-editing memory in a transformer," in *International Conference on Learning Representations*, 2023.
- [37] M. Geva, R. Schuster, J. Berant, and O. Levy, "Transformer feed-forward layers are key-value memories," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 5484–5495.
- [38] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, "Locating and editing factual associations in GPT," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 17 359–17 372.
- [39] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.
- [40] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. Li *et al.*, "Deepseek-coder: when the large language model meets programming—the rise of code intelligence," *arXiv preprint arXiv:2401.14196*, 2024.
- [41] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv *et al.*, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.
- [42] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.
- [43] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM computing surveys*, vol. 55, no. 12, pp. 1–38, 2023.
- [44] A. Alansari and H. Luqman, "Large language models hallucination: A comprehensive survey," *Computer Science Review*, vol. 61, p. 100970, 2026.
- [45] F. Liu, Y. Liu, L. Shi, H. Huang, R. Wang, Z. Yang, L. Zhang, Z. Li, and Y. Ma, "Exploring and evaluating hallucinations in llm-powered code generation," *arXiv e-prints*, pp. arXiv:2404.20404, 2024.
- [46] Y. Tian, W. Yan, Q. Yang, X. Zhao, Q. Chen, W. Wang, Z. Luo, L. Ma, and D. Song, "Codehalu: Investigating code hallucinations in llms via execution-based verification," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 24, 2025, pp. 25 300–25 308.
- [47] A. Sinitin, V. Plokhomyuk, D. Pyrkun, S. Popov, and A. Babenko, "Editable neural networks," *arXiv preprint arXiv:2004.00345*, 2020.
- [48] C. Zhu, A. S. Rawat, M. Zaheer, S. Bhojanapalli, D. Li, F. Yu, and S. Kumar, "Modifying memories in transformer models," *arXiv preprint arXiv:2012.00363*, 2020.
- [49] P. Wang, N. Zhang, B. Tian, Z. Xi, Y. Yao, Z. Xu, M. Wang, S. Mao, X. Wang, S. Cheng *et al.*, "Easyedit: An easy-to-use knowledge editing framework for large language models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, 2024, pp. 82–93.
- [50] E. Mitchell, C. Lin, A. Bosselut, C. D. Manning, and C. Finn, "Memory-based model editing at scale," in *Proceedings of the 39th International Conference on Machine Learning*, 2022, pp. 15 817–15 831.
- [51] T. Hartvigsen, S. Sankaranarayanan, H. Palangi, Y. Kim, and M. Ghassemi, "Aging with GRACE: Lifelong model editing with discrete key-value adapters," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [52] C. Zheng, L. Li, Q. Dong, Y. Fan, Z. Wu, J. Xu, and B. Chang, "Can we edit factual knowledge by in-context learning?" in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 4862–4876.
- [53] N. De Cao, W. Aziz, and I. Titov, "Editing factual knowledge in language models," in *Proceedings of the 2021 conference on empirical methods in natural language processing*, 2021, pp. 6491–6506.
- [54] E. Mitchell, C. Lin, A. Bosselut, C. Finn, and C. D. Manning, "Fast model editing at scale," in *International Conference on Learning Representations*, 2022.
- [55] J.-C. Gu, H.-X. Xu, J.-Y. Ma, P. Lu, Z.-H. Ling, K.-W. Chang, and N. Peng, "Model editing harms general abilities of large language models: Regularization to the rescue," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 16 801–16 819.
- [56] A. Gupta, A. Rao, and G. Anumanchipalli, "Model editing at scale leads to gradual and catastrophic forgetting," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 15 202–15 232.
- [57] S. Liu, X. Hu, K. Huang, X. Yang, D. Lo, and X. Xia, "CREME: Robustness enhancement of code LLMs via layer-aware model editing," in *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, 2026.
- [58] X. Li, S. Wang, S. Li, J. Ma, J. Yu, X. Liu, J. Wang, B. Ji, and W. Zhang, "Model editing for LLMs4code: How far are we?" *arXiv preprint arXiv:2411.06638*, 2024, accepted by ICSE 2025.